ALU Blockchain Track: Final Project Report

Author: Aimable Nkurikiyimana

Abstract

This project develops a decentralized application to securely register and manage the intellectual property of the ALU logo. It utilizes an ERC-721 smart contract to immutably verify the logo's cryptographic hash, preventing counterfeiting, and an ERC-20 smart contract to tokenize ownership into 1,000,000 fractional shares. This system eliminates the need for a centralized database, ensuring transparent authenticity and automated equity distribution.

Introduction

ALU needs its logo protected on a blockchain to guarantee absolute immutability and transparent ownership that cannot be secretly altered or corrupted by a single administrator or a central server failure. While a traditional database can be hacked or modified by anyone with root access, a blockchain secures the logo's cryptographic hash across a decentralized network. Furthermore, standard copyright registration is a slow, manual legal process that does not easily allow for micro-distributions of ownership. This blockchain solution solves these problems by enabling programmable equity, allowing ALU to instantly verify authenticity and distribute fractional ownership worldwide without legal intermediaries.

Blockchain Setup

Hardhat was utilized as the Ethereum development environment for this project. By running a local blockchain node on the development machine, Hardhat simulates a live Ethereum network and provides automated test accounts loaded with simulated ETH. This setup allows for the compilation of Solidity smart contracts, their deployment to a secure local network, and the execution of automated Mocha/Chai tests to mathematically verify the contract logic before pushing the code to a public mainnet.

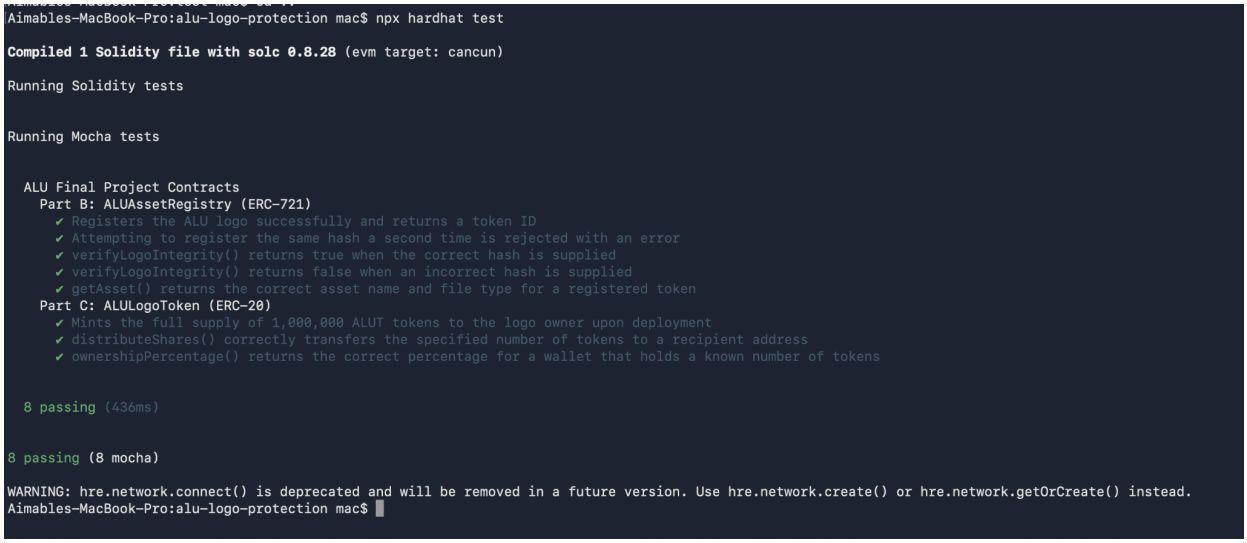
Smart Contract Explanation The project relies on two interconnected smart contracts:

1. ALUAssetRegistry (ERC-721):
    - registerAsset: Takes the logo's name, file type, and cryptographic hash, mints a unique NFT to the caller, and permanently stores the asset's metadata on the blockchain.
    - verifyLogoIntegrity: Compares a user-provided hash against the hash stored on the blockchain to verify if a given file is the authentic ALU logo.
    - getAsset: Retrieves the full metadata (name, file type, hash, and registration time) for a specific token ID.

- *Hash Generation:* The SHA-256 hash generated for the ALU logo is 0x102109d75b09daff117165acb7a2c5d062f252bdab6e9fc83f460b475f957ea5. It was generated in the terminal using a script that mathematically processes the exact pixel data of the alu-logo.png file, ensuring any alteration to the image would result in a completely different hash (the avalanche effect).

2. ALULogoToken (ERC-20):
   - distributeShares: Restricted to the contract owner, this function transfers a specified number of ALUT tokens from the owner's reserve to a recipient's wallet, granting them a fractional share of the logo.
   - ownershipPercentage: Calculates and returns a wallet's ownership stake as a whole percentage by dividing their ALUT token balance by the total supply of 1,000,000.

Test Results



PART A report

What is a local blockchain and why do developers use one instead of the real Ethereum network during development?

A local blockchain (like the Hardhat network used in this project) is a simulated environment running entirely on a developer's computer. Developers use it instead of the real Ethereum mainnet for three primary reasons:

- **Cost:** Deploying contracts and running tests on the real Ethereum network requires spending actual cryptocurrency (ETH) to pay for "gas" fees. A local network provides infinite simulated ETH for free.

- **Speed:** Real blockchain networks take time to process and mine blocks. A local network processes transactions instantly, allowing for rapid testing and debugging.
- **Safety:** Smart contracts are immutable once deployed. A local sandbox allows developers to make mistakes, crash the environment, and wipe the slate clean without permanent consequences or exposing vulnerabilities to the public.

What is a wallet address and what does it represent on the blockchain?

A wallet address is a unique, public string of alphanumeric characters (starting with "0x" on Ethereum) derived from a cryptographic public key. It acts similarly to a bank account number. On the blockchain, it represents a specific user's identity and serves as the definitive location where their digital assets—such as ETH, the ALU logo NFT, or ALUT ownership shares—are securely held and tracked.

PART B Report

1. What is a smart contract? How is it different from a normal programme running on a server? A smart contract is a self-executing program where the logic and rules are written directly into code and deployed onto a decentralized blockchain network. It differs from a normal server program in several critical ways:

- Decentralization: It runs simultaneously across thousands of independent nodes rather than a single centralized server.
- Immutability: Once deployed to the blockchain, the code cannot be altered, patched, or secretly manipulated by the creator.
- Transparency: Both the source code and the historical state of the contract's data are publicly visible to anyone on the network.
- Deterministic: Executing a smart contract function will yield the exact same output across all nodes, guaranteeing an execution consensus without needing a trusted middleman.

2. What is an NFT and why is ERC-721 the right standard for registering a unique asset like a logo? An NFT (Non-Fungible Token) is a digital certificate stored on a blockchain that proves ownership and authenticity of a unique asset. The ERC-721 standard is the correct choice for registering a logo because it forces every token to be unique. Unlike ERC-20 (where every token is identical and interchangeable, like physical coins), ERC-721 assigns a completely distinct tokenId to each asset. This ensures that the ALU logo remains singular and distinct from any other asset registered on the contract.

3. What is a SHA-256 hash and why does changing even one pixel of the logo produce a completely different hash? A SHA-256 hash is a cryptographic algorithm that processes input data of any size and outputs a fixed 256-bit string of hexadecimal characters. Changing even one pixel of the logo

produces a completely different hash because of a cryptographic property known as the "avalanche effect." The algorithm is designed so that even a single-bit alteration in the input file triggers a cascading change throughout the mathematical calculations, resulting in a wildly unpredictable and entirely different final hash, making it an excellent tool for proving file integrity.

PART C REPORT

Tokenisation Tokenization is the process of converting the rights to an asset into digital, fungible tokens on a blockchain, effectively fractionalizing ownership so multiple people can co-own an asset. At ALU, these tokens could be distributed to students as rewards for hackathons or academic excellence, to faculty as performance incentives, and to administration, who would retain majority control. In practice, if a faculty member holds 100,000 ALUT tokens (10% of the supply), they own 10% of the logo's intellectual property. This practically entitles them to 10% of any licensing royalties the logo generates and grants them a 10% voting weight in any decentralized governance decisions regarding how the brand is used commercially.

Tokenisation Tokenization is the process of converting the rights to an asset into digital, fungible tokens on a blockchain, effectively fractionalizing ownership so multiple people can co-own an asset. At ALU, these tokens could be distributed to students as rewards for hackathons or academic excellence, to faculty as performance incentives, and to administration, who would retain majority control. In practice, if a faculty member holds 100,000 ALUT tokens (10% of the supply), they own 10% of the logo's intellectual property. This practically entitles them to 10% of any licensing royalties the logo generates and grants them a 10% voting weight in any decentralized governance decisions regarding how the brand is used commercially.

Conclusion

This project provided profound insights into the architecture of decentralized applications and the intricacies of modern Ethereum development environments. Beyond mastering the core logic of Solidity and utilizing OpenZeppelin libraries to fuse ERC-721 and ERC-20 standards, navigating complex dependency resolutions and ECMAScript Module configurations in Hardhat highlighted the rigorous system management required in blockchain engineering.